



NOTRE DAME UNIVERSITY

BANGLADESH

Computer Graphics Lab Project Report

Course Code: CSE-4204

Course Title: Computer Graphics Lab

Project Title: Egg Drop Saga (Game)

Submitted by:

Name: Istiak Alam

ID: 0692230005101005

Name: Tanvir Hossain Ratul

ID: 0692230005101004

Batch: CSE-20

Submission Date: June 9, 2026

Submitted to:

Humayara Binte Rashid

Lecturer, Dept. of CSE

Notre Dame University Bangladesh

Table of Contents

Abstract	1
1 Introduction	2
1.1 Project Background	2
1.2 Motivation	2
1.3 Objectives	3
1.4 Scope	3
1.5 Tools and Libraries Used	4
2 System Design	5
2.1 Project Structure Overview	5
2.2 System Overview	6
2.3 Game State Management	7
2.4 OpenGL Rendering Pipeline	8
2.5 Coordinate System Design	10
3 Adder Circuits	11
3.1 Half Adder Function	11
4 Flip-Flop Circuits	13
4.1 Function of FlipFlop Circuit	13
5 Stick Diagram in MICROWIND2	15
5.1 1 Input Inverter (NOT Gate)	16
5.2 2 Input NAND Gate	18
5.3 2 Input NOR Gate	20
6 Stick Diagram of XOR function	22
Conclusion	25

Abstract

Egg Drop Saga is a 2D arcade-style game developed using OpenGL and GLUT in C++ as part of the Computer Graphics Laboratory course. The project demonstrates the practical application of fundamental computer graphics concepts including coordinate systems, geometric modeling, transformations, animation, rendering pipelines, and user interaction. In the game, a player controls a bucket to catch eggs dropped by moving chickens while avoiding missed catches that reduce the player's lives. The project incorporates real-time animation, keyboard-based controls, collision detection, score tracking, audio integration, and multiple game states such as menu, gameplay, pause, and game-over screens. The visual elements are created using OpenGL primitive drawing techniques and coordinate-based object construction. The primary objective of the project is to apply theoretical graphics concepts in an interactive environment while gaining experience in event-driven programming and real-time rendering. The final implementation successfully demonstrates the integration of computer graphics principles with game development techniques.

Chapter 1 Introduction

1.1 Project Background

Computer Graphics is a branch of computer science that focuses on the creation, manipulation, and rendering of visual content using computational techniques. OpenGL is one of the most widely used graphics libraries for developing interactive graphical applications and games. As part of the Computer Graphics Laboratory course, students are encouraged to apply theoretical concepts such as coordinate systems, geometric modeling, transformations, animation, rendering, and event handling in a practical project.

The project **Egg Drop Saga** is a 2D arcade-style game developed using C++ and OpenGL (GLUT). The game simulates a farm environment where chickens continuously move and drop eggs from the top of the screen. The player controls a bucket positioned at the bottom of the screen and must catch the falling eggs to earn points while avoiding missed catches that reduce the player's lives. The project integrates graphical object modeling, real-time animation, collision detection, user interaction, and audio feedback into a complete gaming experience.

1.2 Motivation

The inspiration for this project came from classic arcade catching games that emphasize quick reaction and hand-eye coordination. These games are simple in design yet provide an engaging user experience through continuous movement, scoring systems, and increasing difficulty.

From an academic perspective, the project offered an opportunity to implement various computer graphics concepts learned throughout the course in a practical environment. Developing a complete game using OpenGL allowed the team to gain hands-on experience with rendering pipelines, object-oriented design, animation techniques, coordinate-based drawing, and event-driven programming while creating an enjoyable interactive application.

1.3 Objectives

The primary objectives of the Egg Drop Saga project are:

- To develop a fully functional 2D game using OpenGL and GLUT.
- To understand and implement the OpenGL rendering pipeline.
- To apply coordinate-based geometric modeling for creating game objects such as chickens, eggs, buckets, and backgrounds.
- To implement real-time animation for moving objects within the game environment.
- To develop keyboard-based user interaction for controlling game elements.
- To implement collision detection between falling eggs and the player's bucket.
- To create a score and life management system.
- To integrate sound effects and background music for enhanced gameplay.
- To gain practical experience in object-oriented game development using C++.

1.4 Scope

The scope of the project includes the design and implementation of a complete 2D arcade game using OpenGL. The project covers graphical object creation, rendering, animation, collision detection, score management, sound integration, and user interaction through keyboard controls.

The game contains multiple functional components including a main menu, gameplay screen, pause system, score tracking, life management, and game-over interface. All visual elements are generated using OpenGL primitives and coordinate-based drawing techniques rather than external game engines.

However, the project does not include advanced features such as 3D graphics, online multiplayer support, artificial intelligence opponents, physics engines, networking systems, or database integration. The focus remains on demonstrating core computer graphics principles within a 2D interactive environment.

1.5 Tools and Libraries Used

The development of Egg Drop Saga required several software tools and libraries, which are listed below:

Tool/Library	Purpose
C++	Primary programming language used for game development.
OpenGL	Graphics rendering library used for drawing and displaying visual objects.
GLUT (OpenGL Utility Toolkit)	Provides window management, event handling, keyboard input, and rendering callbacks.
SDL2_mixer	Used for audio playback including background music and sound effects.
Code::Blocks	Integrated Development Environment (IDE) used for coding and debugging.
MinGW GCC Compiler	Compiles and links the C++ source code.
Windows Operating System	Development and testing platform.
GitHub	Version control and project collaboration.

Table 1.1: Tools and Libraries Used

The combination of these tools enabled the development of an interactive OpenGL game while providing efficient debugging, code management, graphics rendering, and audio integration capabilities.

Chapter 2 System Design

2.1 Project Structure Overview

The Egg Drop Saga project follows a modular object-oriented architecture where each major game component is implemented as an independent class. This design improves code maintainability, readability, and scalability by separating rendering, game logic, audio management, and user interface functionalities into distinct modules.

The overall project directory structure is shown below:

```
Egg-Drop-Saga
  assets
    egg_drop_saga.ico
    icon.rc
    Linker Setting.txt
    sdl2-config
    sounds
      bucket-catch.wav
      chickenpokpok.wav
      egg-broke.wav
      gamemusic.mp3
      gameover_music.mp3
      home_music.mp3

  include
    AudioManager.h
    Background.h
    Bucket.h
    Chicken.h
    Egg.h
    Game.h
    Home.h

  src
    AudioManager.cpp
    Background.cpp
    Bucket.cpp
    Chicken.cpp
    Egg.cpp
    Game.cpp
```

Home.cpp

main.cpp

Project Configuration Files

The purpose of each major component is described below:

Module	Responsibility
main.cpp	Entry point of the application. Initializes OpenGL, creates the game window, registers callback functions, and starts the GLUT event loop.
Game Class	Controls the overall game state, score system, life management, collision detection, rendering coordination, and gameplay flow.
Chicken Class	Creates and animates the chickens that move horizontally and drop eggs.
Egg Class	Manages egg creation, falling animation, collision checking, and respawning logic.
Bucket Class	Represents the player-controlled bucket used to catch falling eggs.
Background Class	Draws the complete game environment including sky, ground, scenery, decorative objects, and visual layers.
Home Class	Handles the home screen, menu interface, instructions, and navigation before gameplay begins.
AudioManager Class	Controls background music, sound effects, audio loading, playback, and stopping mechanisms using SDL2_mixer.
Assets Directory	Stores sound effects, music files, application icons, and additional project resources.

Table 2.1: Project Modules and Responsibilities

The modular architecture ensures that graphical rendering, user interaction, animation, collision detection, and audio processing remain logically separated, making the project easier to understand, debug, and extend in future versions.

2.2 System Overview

Egg Drop Saga is implemented using an event-driven architecture based on OpenGL and GLUT. The system is organized around a central Game class that manages gameplay logic, rendering operations, collision detection, animation updates, score management, and state transitions.

The application begins execution from main.cpp, where the OpenGL environment, GLUT window, audio system, and callback functions are initialized. Once initialization is complete, GLUT takes control through its event loop and continuously invokes registered callback functions.

The overall execution flow is shown below:

```
main()
|
+--> OpenGL Initialization
|
+--> Audio Initialization
|
+--> Game Initialization
|
+--> Register GLUT Callbacks
|
+--> Enter GLUT Main Loop
|
+--> Display Callback
|
+--> Timer Callback
|
+--> Keyboard Callback
|
+--> Special Key Callback
|
+--> Reshape Callback
```

The system continuously updates and renders the game at approximately 60 frames per second using a timer-based animation loop.

2.3 Game State Management

The game operates using a finite state machine (FSM) implemented through the `GameState` enumeration.

```
enum GameState
{
    HOME,
    PLAYING,
    GAME_OVER
};
```

The state transitions occur as follows:

```
HOME
|
| Press S
v
PLAYING
|
| Lives = 0
v
GAME_OVER
```

|
| Press R
v
HOME

This approach simplifies gameplay management by ensuring that only the logic associated with the active state is executed.

2.4 OpenGL Rendering Pipeline

The rendering process of Egg Drop Saga follows a structured pipeline that is executed repeatedly during gameplay.

Stage 1: Initialization

During program startup, the following operations are performed:

- GLUT initialization using `glutInit()`.
- Creation of the game window.
- Activation of RGB color mode.
- Activation of double buffering.
- Configuration of alpha blending.
- Setup of the orthographic projection.
- Initialization of game resources.
- Loading of audio files.

Stage 2: Input Processing

Player input is captured through GLUT callback functions.

- `keyboard()` handles standard keyboard input.
- `specialKeys()` handles arrow key input.

The input is forwarded to:

```
game.handleInput()  
game.handleSpecialInput()
```

These functions update the bucket position, game state, pause state, and audio settings.

Stage 3: Update Cycle

The timer callback executes every 16 milliseconds.

```
glutTimerFunc(16, timer, 0);
```

This produces approximately:

$$FPS = \frac{1000}{16} \approx 60$$

During each update cycle:

- Background animation is updated.
- Home screen animation is updated.
- Eggs are spawned if necessary.
- Egg positions are updated.
- Collision detection is performed.
- Score and life counters are updated.

Stage 4: Rendering

The display callback invokes:

```
game.render();
```

The rendering order is:

1. Clear frame buffer.
2. Draw background.
3. Draw chickens.
4. Draw eggs.
5. Draw bucket.
6. Draw score text.
7. Draw life indicators.
8. Draw state-specific screens.

Stage 5: Buffer Swapping

After rendering completes:

```
glutSwapBuffers();
```

The back buffer becomes visible while the next frame is rendered in the hidden buffer.

This double-buffering technique eliminates screen flickering and ensures smooth animation.

2.5 Coordinate System Design

Egg Drop Saga uses a two-dimensional orthographic coordinate system configured through:

```
gluOrtho2D(0, GAME_WIDTH, 0, GAME_HEIGHT);
```

The coordinate system is defined as follows:

- Origin (0,0) is located at the bottom-left corner.
- Positive X values extend toward the right.
- Positive Y values extend upward.
- All game objects are positioned using world coordinates.

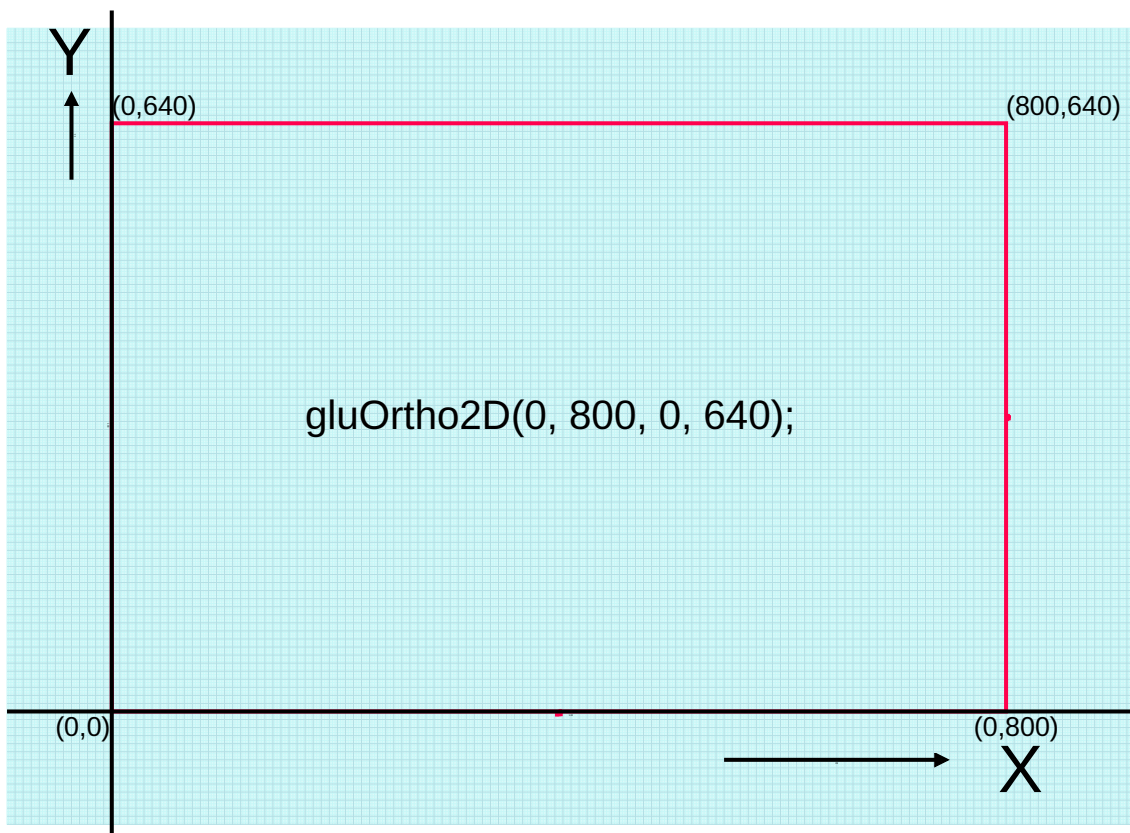


Figure 2.1: orthographics Graph of the Game Screen

Examples of object positioning:

- Bucket is initialized near the bottom of the screen.
- Chickens are positioned along the upper wire.
- Eggs spawn below randomly selected chickens.
- UI elements are positioned using fixed coordinates.

Chapter 3 Adder Circuits

Introduction

A combinational logic circuit which is designed to add two binary digits is called as a half adder. The half adder provides the output along with a carry value (if any). The half adder circuit is designed by connecting an EX-OR gate and one AND gate. It has two input terminals and two output terminals for sum and carry.

Objective

The objectives of this VLSI-Lab experiment are as follows:

- To implement Adder Circuit using CMOS logic design methodology.
- To design schematic-level Adder circuits using the DSCH2 tool.
- To verify the logical correctness of the implemented circuits using truth tables.
- To analyze timing diagrams to observe signal transitions and propagation behavior.
- To develop a clear understanding of Adder Circuit.

3.1 Half Adder Function

Boolean Function

$$Sum = AB' + A'B$$

$$Carry = A B$$

Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Schematic Image

Figure 3.1: CMOS Schematic of Half-Adder

Timing Diagram

Figure 3.2: Timing Diagram of Half-Adder

Observation

The timing diagram of a half adder illustrates the sequence of operations that occur when two binary inputs are added. The diagram typically shows the timing of the XOR operation for the sum and the timing of the AND operation for the carry. The timing diagram can help in understanding the propagation delay and the timing constraints of the circuit. It is important to note that the timing diagram should be adjusted to account for the specific timing requirements of the circuit design, such as rise/fall time and maximum input frequency.

Chapter 4 Flip-Flop Circuits

Introduction

This laboratory experiment focuses on the design and implementation of a flip-flop circuit using CMOS logic in the DSCH2 environment. Flip-flops are fundamental sequential elements used for data storage and synchronization in digital systems. In this experiment, a D Flip-Flop was designed using CMOS-based logic gates, and its functionality was verified through truth table analysis and timing simulation. The behavior of the circuit was observed with respect to clock-triggered input transitions.

Objective

The objectives of this laboratory experiment are as follows:

- To design and implement a flip-flop circuit using CMOS logic.
- To develop schematic-level circuits using the DSCH2 tool.
- To derive and verify the truth table of the flip-flop from simulation.
- To analyze timing diagrams to observe clock-triggered behavior.
- To understand the operation of sequential circuits and data storage elements.

4.1 Function of FlipFlop Circuit

The D Flip-Flop is a clock-controlled bistable device that transfers the input data (D) to the output (Q) on the active edge of the clock signal.

$$Q_{next} = D$$

The output changes only at the triggering edge of the clock, ensuring synchronized data storage.

Truth Table

Clock	D	Q(next)
↑	0	0
↑	1	1
No Edge	X	Q(previous)

Schematic Image

Figure 4.1: CMOS Schematic of D Flip-Flop in DSCH2

Timing Diagram

Figure 4.2: Timing Diagram of D Flip-Flop

Observation

The timing diagram shows that the output Q follows the input D only at the rising edge of the clock signal. Between clock edges, the output remains constant regardless of changes in the input.

From the simulation results, it was observed that the D Flip-Flop changes its output state only at the rising edge of the clock signal. When the clock transitions from LOW to HIGH, the output Q takes the value of the input D. At all other times, the output remains unchanged, preserving the previously stored value. This behavior confirms the correct operation of the edge-triggered D Flip-Flop implemented using CMOS logic.

Chapter 5 Stick Diagram in MICROWIND2

Introduction

In VLSI design, the transition from schematic-level representation to physical layout is a critical step in the design process. Stick diagrams serve as an intermediate abstraction that simplifies the visualization of circuit layouts without considering exact dimensions. These diagrams use standardized color codes to represent different layers such as polysilicon, diffusion regions, and metal interconnections.

Microwind2 is a layout design tool that allows designers to implement and simulate CMOS circuits at the physical level. In this experiment, stick diagrams of basic logic gates were implemented to understand layout organization, transistor placement, and interconnection strategies. The study of stick diagrams helps in developing an understanding of design rules, layout optimization, and area-efficient circuit implementation in VLSI systems.

Objective

The objectives of this laboratory experiment are as follows:

- To understand the concept and importance of stick diagrams in VLSI layout design.
- To implement stick diagrams of basic CMOS logic gates using Microwind2.
- To represent transistor-level circuits using simplified, technology-independent layouts.
- To analyze the arrangement of polysilicon, diffusion, and metal layers in CMOS design.
- To develop a clear understanding of the relationship between schematic design and physical layout.

5.1 1 Input Inverter (NOT Gate)

The inverter, also known as a NOT gate, is the most fundamental logic gate in digital design. It produces the logical complement of the input signal. In CMOS technology, the inverter is implemented using one PMOS transistor in the pull-up network and one NMOS transistor in the pull-down network. The stick diagram represents this structure using polysilicon for gate control, diffusion regions for source and drain, and metal layers for interconnections. The inverter serves as the basic building block for more complex CMOS circuits.

Stick Diagram

Figure 5.1: Stick Diagram of Inverter (NOT Gate) in MICROWIND2

Output

Figure 5.2: Analog Simulation of Inverter (NOT Gate)

Observation

From the output waveform, it is observed that the inverter produces the exact complement of the input signal. When the input is at logic HIGH, the NMOS transistor conducts and pulls the output to logic LOW. Conversely, when the input is at logic LOW, the PMOS transistor conducts and pulls the output to logic HIGH. The transition between output states occurs immediately following the input change, demonstrating correct switching behavior. The observed output confirms the proper operation of the CMOS inverter as expected from its theoretical truth table.

5.2 2 Input NAND Gate

A 2-input NAND gate is a fundamental universal logic gate that produces a logic LOW output only when both inputs are logic HIGH. In CMOS layout design, the NAND gate is implemented using two NMOS transistors connected in series in the pull-down network and two PMOS transistors connected in parallel in the pull-up network. The stick diagram represents this structure using simplified layer representations such as polysilicon for gates, diffusion for active regions, and metal for interconnections.

Stick Diagram

Figure 5.3: Stick Diagram of 2in-NAND Gate in MICROWIND2

Output

Figure 5.4: Analog Simulation of 2in-NAND Gate

Observation

From the simulation results obtained in Microwind2, it is observed that the output remains at logic HIGH for all input combinations except when both inputs are HIGH. When inputs A and B are simultaneously at logic HIGH, the series NMOS transistors in the pull-down network conduct, creating a path to ground and forcing the output to logic LOW. For all other input conditions, at least one PMOS transistor in the pull-up network remains active, maintaining the output at logic HIGH. The observed output waveform matches the expected truth table of a 2-input NAND gate, confirming the correctness of the stick diagram implementation.

5.3 2 Input NOR Gate

The 2-input NOR gate is a fundamental digital logic gate that produces a logic HIGH output only when both inputs are at logic LOW. In CMOS implementation, the NOR gate is realized using parallel NMOS transistors in the pull-down network and series PMOS transistors in the pull-up network. In the stick diagram representation using Microwind2, different layers such as polysilicon, diffusion, and metal are arranged according to standard layout conventions to form the required transistor connections. The stick diagram provides a simplified visualization of the physical layout without considering exact dimensions.

Stick Diagram

Figure 5.5: Stick Diagram of 2in-NOR Gate in MICROWIND2

Output

Figure 5.6: Analog Simulation of 2in-NOR Gate

Observation

From the simulation results, it is observed that the output of the 2-input NOR gate remains logic HIGH only when both inputs are LOW (0,0). For all other input combinations, i.e., when any one or both inputs are HIGH, the output becomes logic LOW. The output waveform clearly reflects this behavior, showing a HIGH level only in the absence of input signals. This verifies the correct implementation of the NOR logic function in the stick diagram using Microwind2.

Chapter 6 Stick Diagram of XOR function

Introduction

In VLSI design, the transformation of logical circuits into physical layouts is an essential step in the design process. Stick diagrams serve as a simplified and technology-independent method for representing circuit layouts, allowing designers to visualize transistor placement and interconnections without focusing on exact dimensions.

The XOR gate is an important combinational logic circuit widely used in arithmetic and digital systems. Its implementation in CMOS requires careful arrangement of pull-up and pull-down networks. In this experiment, the XOR gate is designed using its Boolean expression and represented using a stick diagram in Microwind2. The study of this design helps in understanding layout organization, layer interaction, and efficient circuit realization in VLSI systems.

Objective

The objectives of this laboratory experiment are as follows:

- To understand the concept and importance of stick diagrams in VLSI layout design.
- To implement stick diagrams of basic CMOS logic gates using Microwind2.
- To represent transistor-level circuits using simplified, technology-independent layouts.
- To analyze the arrangement of polysilicon, diffusion, and metal layers in CMOS design.
- To develop a clear understanding of the relationship between schematic design and physical layout.

An XOR (Exclusive-OR) gate is a combinational logic gate that produces a logic HIGH output when the inputs are different and a logic LOW output when the inputs are the same. It is commonly used in digital circuits for comparison, parity checking, and arithmetic operations.

Boolean Function of XOR Gate

$$Y = (AB + A'B')'$$

Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Stick Diagram

Figure 6.1: Stick Diagram of XOR Gate

Output

Figure 6.2: Output in Analog Simulation of XOR Gate

Observation

From the implementation and simulation of the XOR gate stick diagram using Microwind2, it was observed that the circuit produces a logic HIGH output when the inputs are different and a logic LOW output when both inputs are the same. The output waveform obtained from the simulation matches the theoretical truth table of the XOR function.

The stick diagram correctly represents the CMOS structure using appropriate placement of polysilicon, diffusion, and metal layers. Proper alignment and interconnection of these layers ensured correct signal flow and circuit operation. The simulation results confirm that the designed layout accurately implements the XOR logic function.

Conclusion

In this VLSI Design Lab course, various CMOS-based digital circuits were successfully designed, implemented, simulated, and analyzed using DSCH2 and MICROWIND2 software tools. The experiments provided practical knowledge of combinational logic circuits, sequential circuits, timing analysis, CMOS implementation techniques, and physical layout representation in VLSI systems.

The implementation of basic logic gates such as NOT, NAND, NOR, AND, OR, and XOR using DSCH2 helped in understanding the operation of CMOS pull-up and pull-down networks. Verification through truth tables and timing simulations confirmed the correct logical behavior of each circuit. The implementation of Boolean expressions further strengthened the understanding of CMOS logic design methodology and transistor level circuit realization.

The design and analysis of Half Adder and D Flip-Flop circuits introduced important concepts of arithmetic operations, sequential logic, data storage, and clock-triggered behavior. Timing diagrams demonstrated signal transitions and propagation characteristics of digital circuits.

The implementation of stick diagrams using MICROWIND2 provided practical experience in translating schematic-level circuits into simplified physical layouts. The arrangement of polysilicon, diffusion, and metal layers helped in understanding transistor placement, routing techniques, and layout organization in CMOS design. Analog simulation outputs verified the correctness of the implemented layouts.

Overall, this laboratory course successfully developed fundamental knowledge and practical skills related to VLSI circuit design, CMOS implementation, timing analysis, and physical layout representation. The experiments provided a strong foundation for advanced study and research in VLSI system design and semiconductor technology.